《现代C++编程实现》电子版：https://leanpub.com/c01

控制台游戏-Pong游戏

YouTube频道： hwdong

博客： hwdong-net.github.io

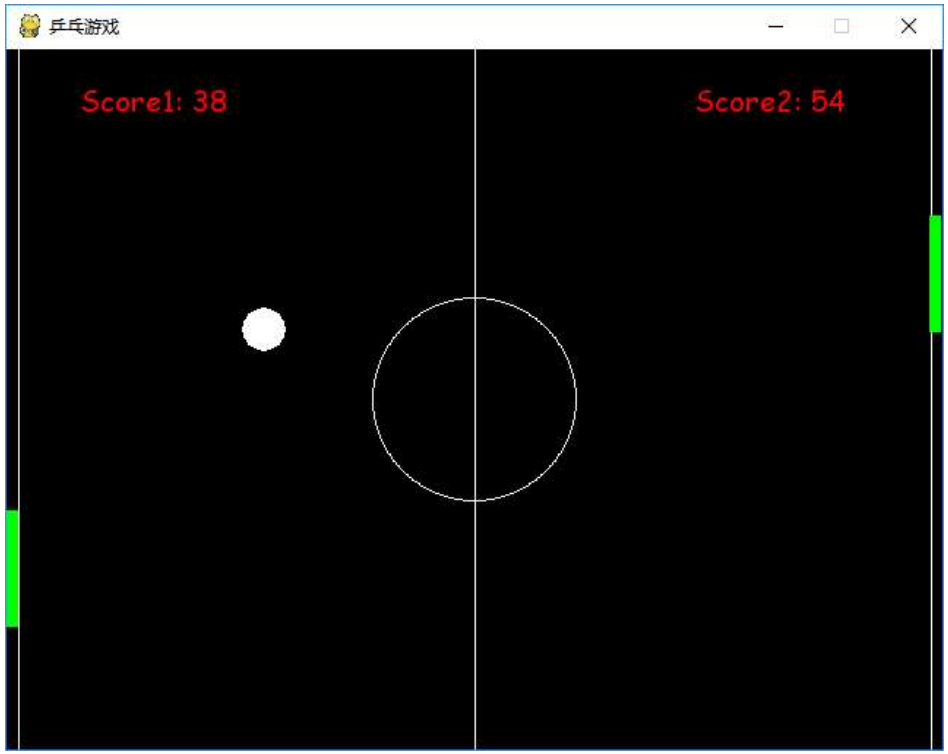
B站: hw-dong

Pong游戏

- 1972年，艾尔·奥尔康（Al Alcorn）为雅达利（Atari）公司开发的模拟乒乓球的《PONG》街机游戏。
- 从酒吧开始风靡美国，很快传到日本。
- 日本的娱乐公司Taito开发了一个类似的克隆产品《Elepong》，成为了日本的第一款电子游戏。

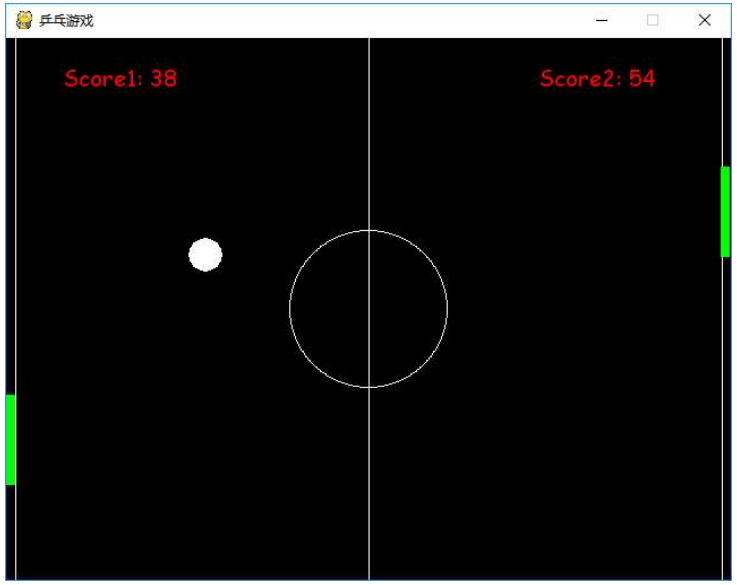
Pong游戏画面

- 1个乒乓球（Ball）和左右两个挡板（Paddle）



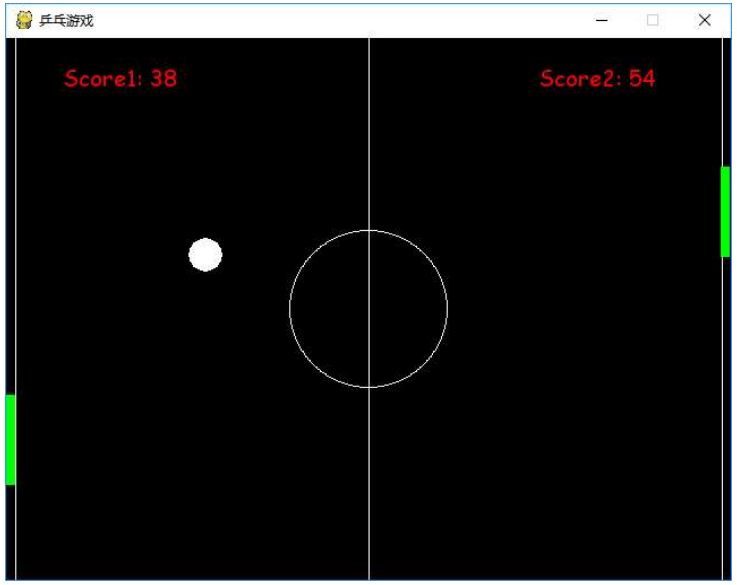
初始化

- 游戏画面有一个窗口，窗口里除了一些背景（如Pong游戏中的分隔线），主要运动物体是一只球（Ball）和2个挡板（Paddle），分别用1个圆和2个矩形表示。



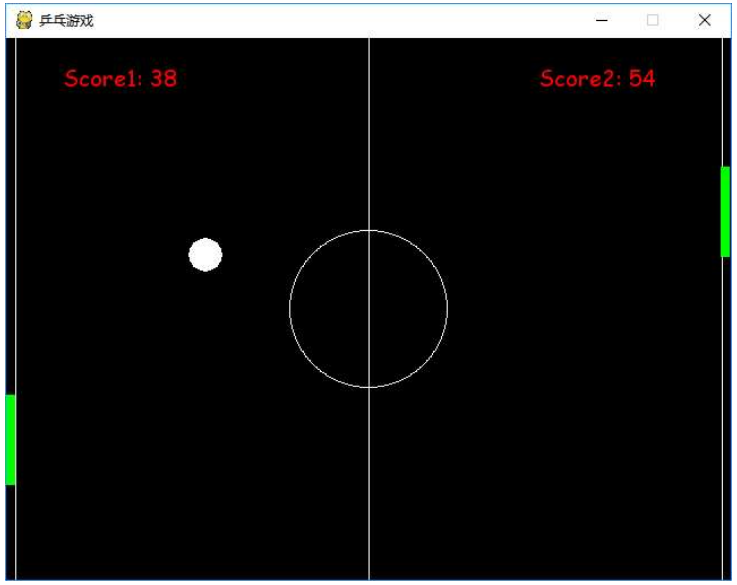
初始化

- 球有位置、大小（半径）、颜色、速度等属性，而挡板也有位置、大小（长宽）、颜色、速度等属性。

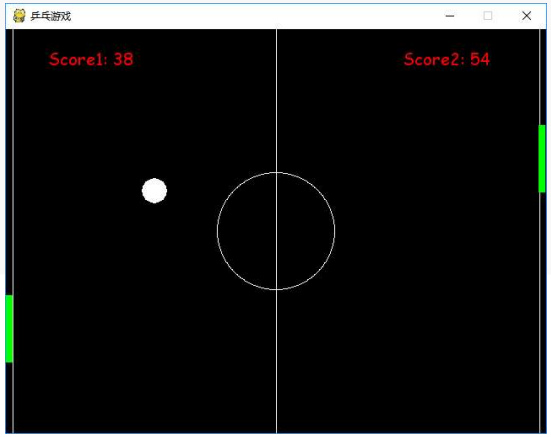


初始化

- 游戏还有记录各自分数的变量。当然，游戏窗口也有长宽、标题、背景颜色等属性。

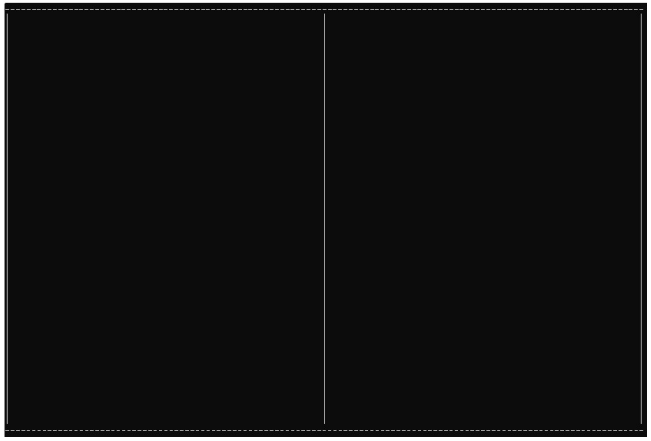


```cpp
int main() {
    // 1. 初始化游戏中的数据
    const auto WIDTH{ 120 }, HEIGHT{ 40 };                  // 窗口长宽
    auto ball_x{ WIDTH / 2 }, ball_y{ HEIGHT / 2 };         // 球的位置
    auto ball_vec_x{ 0 }, ball_vec_y{ 0 };                  // 球的速度分量
    const auto paddle_w{ 3 }, paddle_h{ 10 };               // 挡板的宽高
    auto paddle1_x{ 0 };                                    // 左挡板x坐标固定为0
    auto paddle1_y{ HEIGHT / 2 - paddle_h / 2 };            // 左挡板y坐标
    auto paddle2_x{ WIDTH - paddle_w };                     // 右挡板x坐标固定为右边界减宽度
    auto paddle2_y{ HEIGHT / 2 - paddle_h / 2 };            // 右挡板y坐标
    const auto paddle_vec{ 3 };                             // 挡板移动速度
    auto score1{ 0 }, score2{ 0 };                          // 双方的得分

    // 后续步骤继续...
    return 0;
}
```



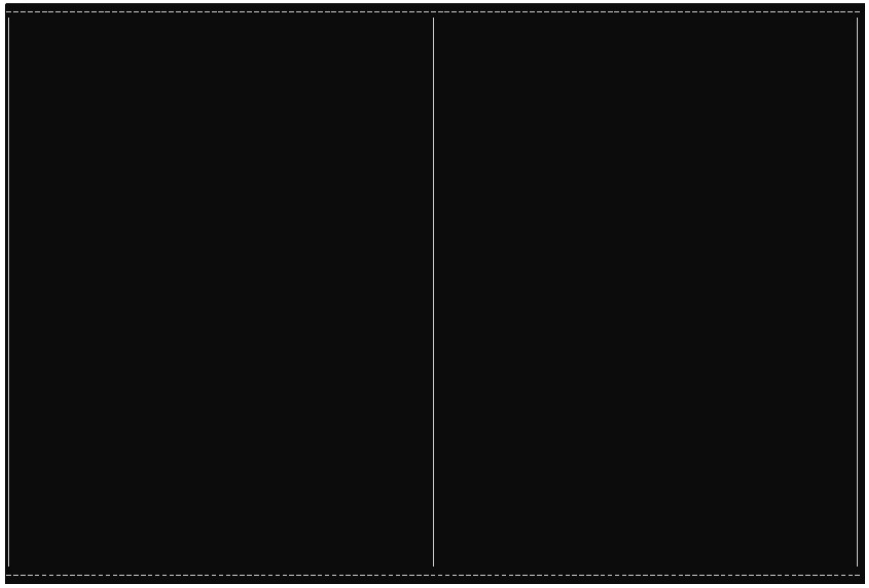
绘制场景

- 绘制场景包括**绘制背景**和游戏中的**运动物体**。
- 可以先绘制背景中的上下墙壁和3条竖线。
- 也就是通过在控制台窗口中输出一些特殊字符如|或=分别表示3条竖线和墙的图案。



绘制背景　上下墙壁和3条竖线

```
// 2.1.1 绘制顶部墙
for (auto x = 0; x < WIDTH; x++)
    std::cout << '=';
std::cout << '\n';

// 2.1.2 绘制背景中的竖线
for (auto y = 0; y < HEIGHT; y++) {
    for (auto x = 0; x < WIDTH; x++)
        if (x == 0 || x == WIDTH / 2 || x == WIDTH-1)
            std::cout << '|';
        else std::cout << ' ';
    std::cout << '\n';
}

// 2.1.3 绘制底部墙
for (auto x = 0; x < WIDTH; x++)
    std::cout << '=';
std::cout << '\n';
```



添加球和挡板

- 挡板可以用一个矩形表示，球用大写字母O表示。

添加球和挡板

- 为了将球和挡板绘制在窗口画面中，需要在前面的绘制背景的循环中判断哪些位置是球和挡板，然后在这些位置绘制代表球和挡板的特殊图案字符。

```
// 2.1.2 绘制背景中的竖线
for (auto y = 0; y < HEIGHT; y++) {
    for (auto x = 0; x < WIDTH; x++)
        if (x == 0 || x == WIDTH / 2 || x == WIDTH-1)
            std::cout << '|';
        else std::cout << ' ';
    std::cout << '\n';
}
```

添加球和挡板

- 修改前面的if...else语句，在球和挡板的位置绘制球和挡板。

```
for (auto y = 0; y < HEIGHT; y++) {
    for (auto x = 0; x < WIDTH; x++) {
        if (x == ball_x && y == ball_y)                          // 画球
            std::cout << 'O';
        else if (y >= paddle1_y && y < paddle1_y + paddle_h
                 && x >= paddle1_x && x < paddle1_x + paddle_w)  // 左挡板
            std::cout << 'Z';
        else if (y >= paddle2_y && y < paddle2_y + paddle_h
                 && x >= paddle2_x && x < paddle2_x + paddle_w)  // 右挡板
            std::cout << 'Z';
        else if (x == 0 || x == WIDTH / 2 || x == WIDTH-1)  // 竖线
            std::cout << '|';
        else std::cout << ' ';
    }
    std::cout << '\n';
}
```

让球动起来

- 游戏中是通过不断绘制新的画面让游戏画面动起来的。
- 游戏实际是如下的一个循环过程，

```
初始化游戏数据
循环（直到游戏结束）{
    处理事件（如用户输入、定时器）
    更新游戏状态（游戏中的数据）
    绘制游戏画面
}
```

游戏代码结构

```
int main() {
    // 初始化游戏数据
    while (true) {
        //处理事件（如玩家输入）
        //更新游戏状态（更新球的位置）
        //绘制游戏画面
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
    return 0;
}
```

```
sleep_for(std::chrono::milliseconds(100));
```

- 人为增加每帧之间的间隔时间，可以在一定程度上控制游戏的刷新率。

例如，每次循环后等待 100 毫秒，相当于让游戏的帧率保持在 10 FPS（每秒 10 帧）。

sleep_for(std::chrono::milliseconds(100));

- 人为增加每帧之间的间隔时间，可以在一定程度上控制游戏的刷新率。

例如，每次循环后等待 100 毫秒，相当于让游戏的帧率保持在 10 FPS（每秒 10 帧）。

```
#include <thread>  // 提供 std::this_thread::sleep_for()
#include <chrono>  // 提供 std::chrono::milliseconds
```

隐藏光标

光标可见，会带来以下问题：

1. **光标不断闪烁**，影响游戏体验。
2. **光标可能干扰游戏元素**（特别是在移动对象时）。
3. **某些终端可能会自动移动光标，导致意外的绘制问题。**

如何隐藏光标?

Linux / macOS 终端

使用 ANSI 转义序列 `\033[?25l` 隐藏光标，`\033[?25h` 重新显示光标。

```
std::cout << "\033[?25l";  // 隐藏光标
std::cout << "\033[?25h";  // 显示光标
```

如何隐藏光标?

Windows 不能直接用 ANSI 代码（除非启用 VT 终端支持），所以用 **WinAPI** 操作光标。

```
#include <windows.h>
#include <iostream>
#include <thread>
#include <chrono>

void hide_cursor() {
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(console, &cursorInfo);
    cursorInfo.bVisible = FALSE;  // 隐藏光标
    SetConsoleCursorInfo(console, &cursorInfo);
}
```

如何显示光标?

程序退出游戏后，可以通过下面的WinAPI调用恢复光标

```
void show_cursor() {
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(console, &cursorInfo);
    cursorInfo.bVisible = TRUE;  // 显示光标
    SetConsoleCursorInfo(console, &cursorInfo);
}
```

```
#ifdef _WIN32
#include <windows.h>

void hide_cursor() {
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(console, &cursorInfo);
    cursorInfo.bVisible = FALSE;
    SetConsoleCursorInfo(console, &cursorInfo);
}

void show_cursor() {
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(console, &cursorInfo);
    cursorInfo.bVisible = TRUE;
    SetConsoleCursorInfo(console, &cursorInfo);
}
```

```
#else
#include <iostream>

void hide_cursor() {
    std::cout << "\033[?25l";
    std::cout.flush();  // 确保立即输出
}

void show_cursor() {
    std::cout << "\033[?25h";
    std::cout.flush();
}
#endif
```

隐藏光标与显示光标

在游戏开始前隐藏光标、游戏解释时恢复显示光标。

```
int main() {
    hide_cursor();  // 启动时隐藏光标

    for (int i = 0; i < 10; ++i) {
        std::cout << "游戏运行中: " << i << "\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }

    show_cursor();  // 退出前恢复光标
    return 0;
}
```

画面重绘：如何清除原先的画面？

在游戏循环中，需要不断绘制新的画面，而新的画面必须覆盖掉旧的画面，以确保视觉上的连续性。

- 清屏操作
- 光标回到 (0,0)。

```
#ifdef _WIN32
        // 定位到 (0,0)，相当于清空屏幕
        SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), COORD{0,0});
#else
        std::cout << "\033[H";  // 将光标移动到 (0,0)
#endif
```

让球动起来：给球一个随机的初始速度

- **现代C++** 提供了 <random> 库，可生成随机的初始化速度。

```
#include <random>   // 现代C++随机数库

// 初始化随机数生成器
std::random_device rd;                          // 获取真随机种子
std::mt19937 gen(rd());                         // 梅森旋转算法引擎
std::uniform_int_distribution<int> velDist(1, 3);   // 速度大小 1~3
std::uniform_int_distribution<int> dirDist(0, 1);   // 方向：0 或 1

// 生成随机速度（带随机正负号）
ball_vec_x = velDist(gen) * (dirDist(gen) ? 1 : -1);
ball_vec_y = velDist(gen) * (dirDist(gen) ? 1 : -1);
```

让球动起来：给球一个随机的初始速度

- 为了方便，我们通常会将随机速度的生成封装成一个宏，以避免重复代码。

```
#define RANDOM_DIRECTION() (velDist(gen) * (dirDist(gen) ? 1 : -1))
```

这样，上面的两行代码就可以简化为：

```
ball_vec_x = RANDOM_DIRECTION();
ball_vec_y = RANDOM_DIRECTION();
```

让球动起来：根据球的速度更新球位置

- 在游戏循环中根据速度不断更新球的位置，并绘制游戏画面，就能让球动起来。

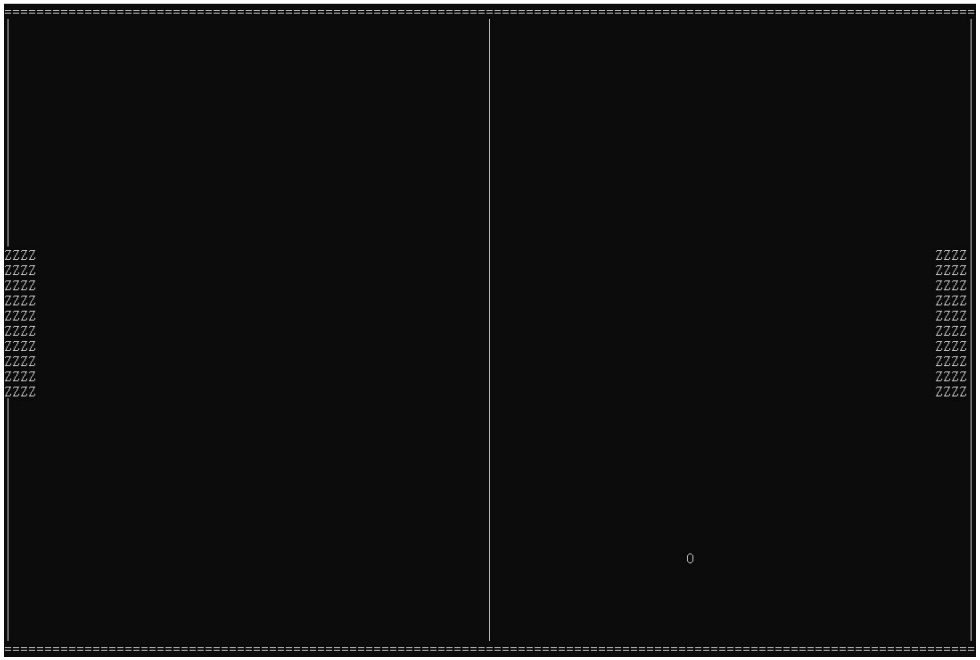
```
ball_x += ball_vec_x;    //根据速度改变位置
ball_y += ball_vec_y;
```

```
// 游戏循环
while (true) {
    // 1. 更新球的位置
    ball_x += ball_vec_x;
    ball_y += ball_vec_y;

#ifdef _WIN32
    // 定位到 (0,0)，相当于清空屏幕
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), COORD{0,0});
#else
    std::cout << "\033[H";  // 将光标移动到 (0,0)
#endif

    // 2. 绘制新画面
    // 绘制顶部墙
    for (auto x = 0; x < WIDTH; x++)
        std::cout << '=';
    std::cout << '\n';
```

```
// 绘制球、挡板和背景
for (auto y = 0; y < HEIGHT; y++) {
    for (auto x = 0; x < WIDTH; x++) {
        if (x == ball_x && y == ball_y)
            std::cout << 'O';
        else if (y >= paddle1_y && y < paddle1_y + paddle_h
                 && x >= paddle1_x && x < paddle1_x + paddle_w)
            std::cout << 'Z';
        else if (y >= paddle2_y && y < paddle2_y + paddle_h
                 && x >= paddle2_x && x < paddle2_x + paddle_w)
            std::cout << 'Z';
        else if (x == 0 || x == WIDTH / 2 || x == WIDTH-1)
            std::cout << '|';
        else std::cout << ' ';
    }
    std::cout << '\n';
}
```

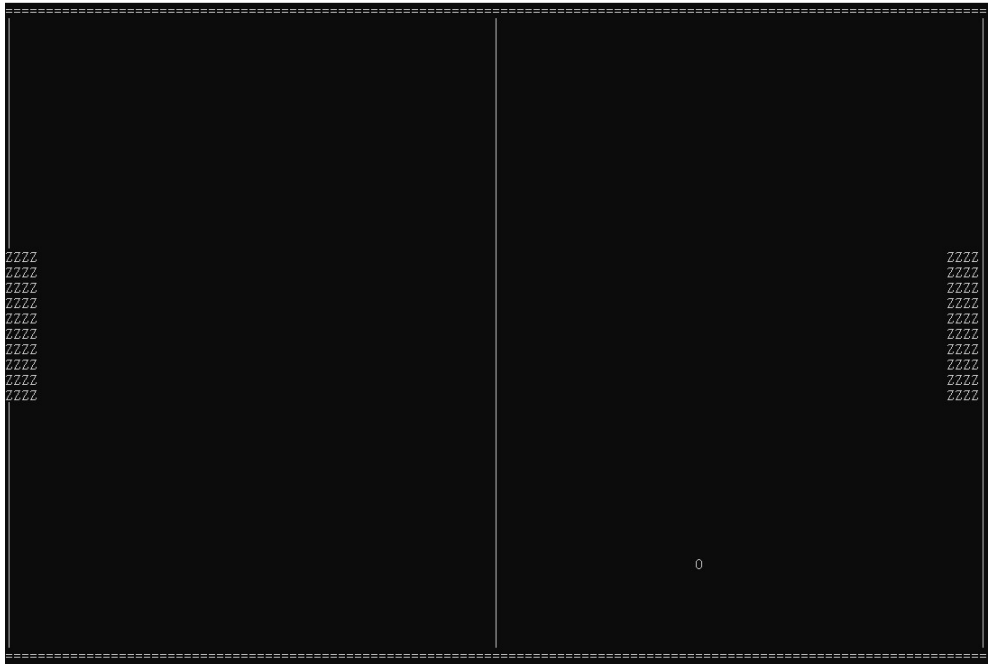
让球动起来：碰撞检测、更新数据

- 球如果和上下墙壁发生碰撞会反弹回来，而遇到挡板也会发生反弹，如果越过左右沟渠而没有遇到挡板，则球跑出画面，另外一方的得分将增加，然后球重新从新位置以随机速度出发。



让球动起来：碰撞检测、更新数据

- 先检测球是否和墙壁或挡板碰撞，如发生碰撞，通过改变球的相应的速度方向，而使球发生反弹，和上下墙壁碰撞，球的垂直速度方向变成反方向，和左右挡板碰撞，球的水平速度方向变成反方向。



让球动起来：碰撞检测、更新数据

```
// 2. 更新数据
ball_x += ball_vec_x;
ball_y += ball_vec_y;

if (ball_y < 0 || ball_y >= HEIGHT) { // 和上下墙碰撞，改变垂直速度方向
    ball_vec_y = -ball_vec_y;
}

if (ball_x < paddle_w && ball_y >= paddle1_y && ball_y < paddle1_y + paddle_h) {
    // 和左挡板碰撞，改变水平速度的方向
    ball_vec_x = -ball_vec_x;
    // 注意：经典Pong规则中，球碰到挡板只反弹，不加分。加分只发生在球越过边界时。
    // 这里去掉score1 += 1，只保留反弹。
} else if (ball_x > WIDTH - paddle_w && ball_y >= paddle2_y && ball_y < paddle2_y + paddle_h) {
    // 和右挡板碰撞，改变水平速度的方向
    ball_vec_x = -ball_vec_x;
}
```

```
// 跑出左右沟渠，球回到中心并以新的随机速度出发，并给对方加分
if (ball_x < 0 || ball_x > WIDTH) {
    if (ball_x < 0) score2++;
    else score1++;
    ball_x = WIDTH / 2;
    ball_y = HEIGHT / 2;
    // 使用现代C++随机数生成新的速度
    ball_vec_x = velDist(gen) * (dirDist(gen) ? 1 : -1);
    ball_vec_y = velDist(gen) * (dirDist(gen) ? 1 : -1);
}
```

让球动起来：事件处理： 用挡板击打球

- 用上下箭头键移动右挡板，而用字母w和s移动左挡板。

让球动起来：事件处理： 用挡板击打球

- 为了得到键盘输入，先用_kbhit()函数检测是否存在按键消息，然后通过_getch()函数得到按键的字符。这2个函数在头文件conio.h中说明。

- 下面代码根据用户输入改变挡板的位置：

```
// 1. 处理事件
char key;
if (_kbhit()) { // 键盘有输入 (注意: _kbhit() 是 Windows 平台相关的函数)
    key = _getch(); // 得到输入的键值 (注意: _getch() 是 Windows 平台相关的函数)

    // 处理左挡板 (Paddle 1) 的移动
    if ((key == 'w' || key == 'W') && paddle1_y > paddle_vec) {
        paddle1_y -= paddle_vec;
    } else if ((key == 's' || key == 'S') && paddle1_y + paddle_vec + paddle_h < HEIGHT) {
        paddle1_y += paddle_vec;
    }

    // 处理右挡板 (Paddle 2) 的移动
    else if (key == 72 && paddle2_y > paddle_vec) { // 72 是向上箭头键的 ASCII 码
        paddle2_y -= paddle_vec;
    } else if (key == 80 && paddle2_y + paddle_vec + paddle_h < HEIGHT) { // 80 是向下箭头键的 ASCII 码
        paddle2_y += paddle_vec;
    }
}
```

让球动起来：如何显示分数?

- 首先定义分数以及显示分数位置的变量：

```
auto score1{ 0 }, score2{ 0 }, score1_x{ paddle_w+8 }, score1_y{ 2 },
                    score2_x{ WIDTH- 8- paddle_w }, score2_y{ 2 };
```

让球动起来：如何显示分数?

- 要将int类型的分数转化为字符串才能通过输出字符串的方式显示分数，可以使用c++标准库的字符串类型string的to_string()函数将一个整数转化为一个字符串。

```
std::string s1{std::to_string(score1)};
std::string s2{std::to_string(score2)};
```

让球动起来：如何显示分数?

- 最后在绘制场景时，在显示分数的位置输出这个表示分数的字符串即可：

```
else if (y == score1_y && x == score1_x) { // 左分数位置
    std::cout << s1;
    while (x < score1_x + s1.size()) {
        x++;
    }
    x--;
} else if (y == score2_y && x == score2_x) { // 右分数位置
    std::cout << s2; //  更正:  应输出 s2，而不是 s1
    while (x < score2_x + s2.size()) {
        x++;
    }

    x--;
}
```

提高程序效率的方法

- 用 std::cout 逐个字符输出游戏画面。虽然代码简单，但每次输出都会与操作系统交互，频繁的 I/O 操作会导致画面刷新缓慢、CPU 占用过高。
- 采用**帧缓冲区**技术：将整个画面预先绘制到内存中的一个字符串中，然后一次性输出。

使用字符串作为帧缓冲区

- 我们需要一个足够大的字符串来存放整个游戏画面。画面由 HEIGHT 行组成，每行 WIDTH 个字符，加上每行末尾的换行符 \n ，因此字符串的长度应为 HEIGHT * (WIDTH + 1) 。

```
// 创建一个字符数组来缓存整个画面（包括换行符）
std::string framebuffer(HEIGHT * (WIDTH + 1), ' ');
```

使用字符串作为帧缓冲区

- 用一个索引 frame_idx 来依次填充字符。例如，将字符 'A' 放入字符串的某个位置：

```
framebuffer[frame_idx++] = '\n';
```

使用字符串作为帧缓冲区

- 用一个索引 frame_idx 来依次填充字符。例如，将字符 'A' 放入字符串的某个位置：

```
framebuffer[frame_idx] = 'A';
frame_idx++;  // 索引递增
```

每填充完一行后，手动添加换行符：

```
framebuffer[frame_idx++] = '\n';
```

使用字符串作为帧缓冲区

- 全部填充完成后，只需一次输出即可显示整个画面：

```
std::cout << framebuffer;
```

```
// 绘制顶部墙
for (auto x = 0; x < WIDTH; x++) {
    framebuffer[frame_idx++] = '=';
}
framebuffer[frame_idx++] = '\n';
```

```
// 绘制球、挡板和背景（跳过顶部和底部墙壁行）
for (auto y = 1, HEIGHT_1 = HEIGHT - 1; y < HEIGHT_1; y++) {
    for (auto x = 0; x < WIDTH; x++) {
        if (x == ball_x && y == ball_y)
            framebuffer[frame_idx++] = 'O';
        else if (y >= paddle1_y && y < paddle1_y + paddle_h
            && x >= paddle1_x && x < paddle1_x + paddle_w)
            framebuffer[frame_idx++] = 'Z';
        else if (y >= paddle2_y && y < paddle2_y + paddle_h
            && x >= paddle2_x && x < paddle2_x + paddle_w)
            framebuffer[frame_idx++] = 'Z';
        else if (y == score1_y && x == score1_x) {  // 左分数位置
            framebuffer[frame_idx++] = s1[0];
            x += s1.size() - 1;
        }
        else if (y == score2_y && x == score2_x) {  // 右分数位置
            framebuffer[frame_idx++] = s2[0];
            x += s2.size() - 1;
        }
        else if (x == 0 || x == WIDTH / 2 || x == WIDTH - 1)
            framebuffer[frame_idx++] = '|';
        else framebuffer[frame_idx++] = ' ';
    }
    framebuffer[frame_idx++] = '\n';
}
```

```
// 绘制底部墙
for (auto x = 0; x < WIDTH; x++) {
    framebuffer[frame_idx++] = '=';
}
framebuffer[frame_idx++] = '\n';

// 输出整个屏幕内容
std::cout << framebuffer;

std::this_thread::sleep_for(std::chrono::milliseconds(50));  // 控制更新速度
```

关注我

博客： hwdong-net.github.io

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links